Lab 1: Design, Implementation, and Simulation of an 8-bit Carry Look Ahead Adder

This lab aims to design, implement, and simulate an 8-bit Carry Look Ahead Adder (CLA) using NAND gates, and other building blocks in a structural, hierarchical style. The delay analysis will be performed to verify timing characteristics.

In Lecture 4, you studied the structure of the Carry Look Ahead Adder (CLA) and examined the SystemVerilog modeling of a 4-bit CLA. You learned how to break down the CLA into smaller modules, such as Propagate and Generate logic, Sum logic, and Carry out logic, as shown in Figure 1, and then integrate them into a complete 4-bit CLA. Now, you will extend this concept by designing an 8-bit CLA using **NAND gates with a fan-in (number of inputs) limited to two, three, and four.**

In your design, you must account for the delays introduced by the different types of NAND gates you use. Below are the delays for NAND gates based on their input count:

- Delay of 2-input NAND = 1.0 T
- Delay of 3-input NAND = 1.25 T
- Delay of 4-input NAND = 1.5 T

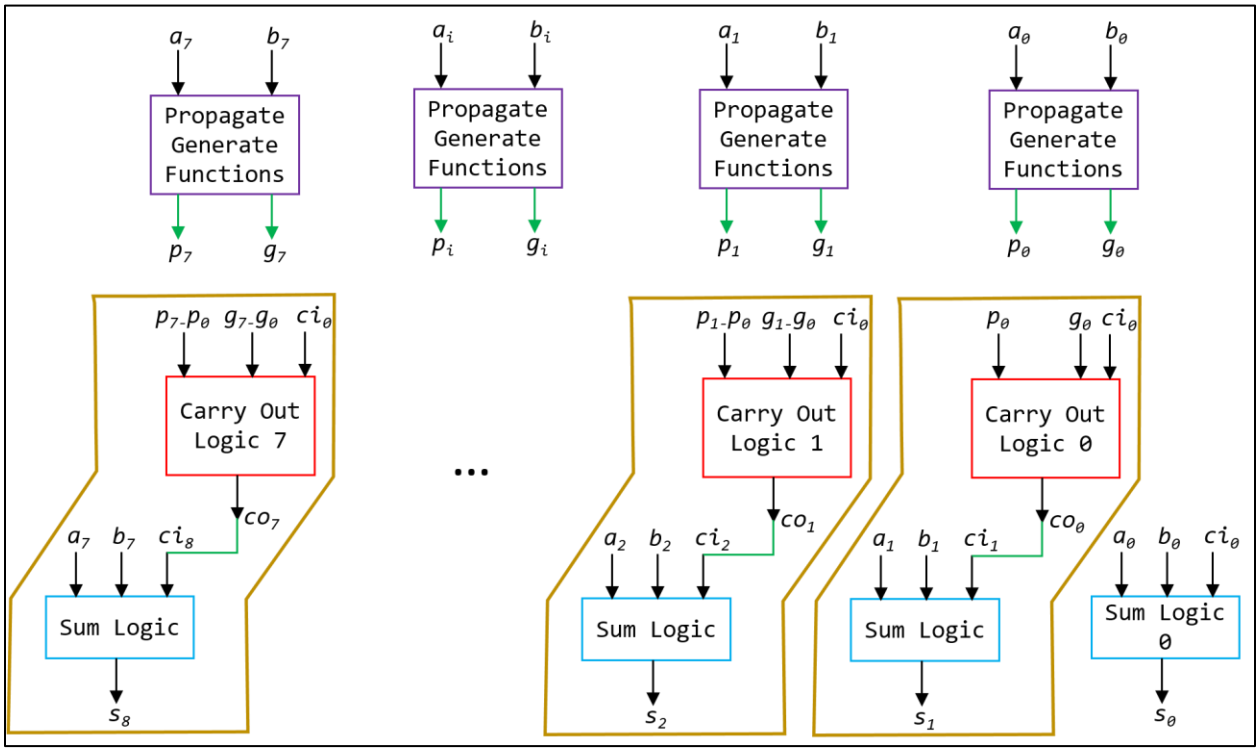


Figure 1– 8-bit Carry Look Ahead Adder (CLA) structure

CLA Calculations:

The 8-bit CLA design will calculate the sum (s_i) and the carry out (co_i) using the following formulas:

Sum Calculation:

$$s_i = a_i \oplus b_i \oplus ci_i$$

Carry Out Calculation:

$$co_i = a_i b_i + ci_i(a_i + b_i);\ ci_i = c_{oi-1}$$

Propagate function:

$$p_i = a_i + b_i$$

Generate function:

$$g_i = a_i b_i$$

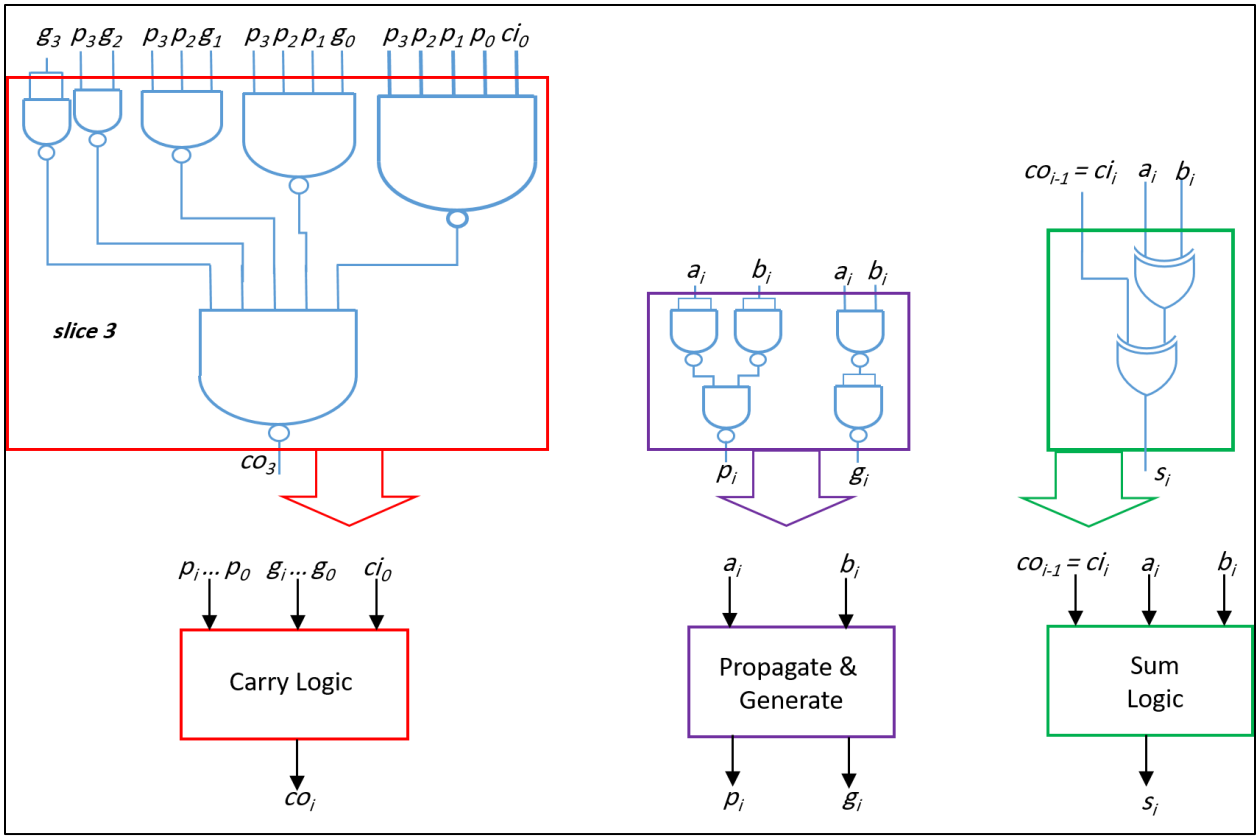


Figure 2– CLA building blocks

Lab steps:

1. **Design and Simulate the Following Modules (Refer to Figure 2 for module structure):**

   The interface of the modules are given below.

   a. **Propagate Logic**: Implements the propagation of the carry signal.

```
module propagate_logic (

      input logic a,

      input logic b,

      output logic p);

//

endmodule
```

   b. **Generate Logic**: Implements the carry generation.

```
module generate_logic (

      input logic a,

      input logic b,

      output logic g);

//

endmodule
```

c. **Sum Logic**: Calculates the sum for each bit.

```
module sum_logic (
    input logic a,
    input logic b,
    input logic carry_in,
    output logic sum);
//
endmodule
```

d. **Carry Out Logic0**: Determines the first carry out based on the carry propagation and generation.

```
module carry_logic_0 (
    input logic p0,
    input logic g0,
    input logic carry_in0,
    output logic carry_out);
//
endmodule
```

2. **Design nandN_delay Gate Modules:** Design and implement NAND gates with 5, 6, 7, 8, and 9 inputs (`nand5_delay`, `nand6_delay`, `nand7_delay`, `nand8_delay`, and `nand9_delay`, respectively) by combining smaller NAND gates (limited to 4 inputs or fewer).

```
module nandN_delay (
    input logic a,   // First input
    input logic b,   // Second input
      ……
    input logic N,   // Nth input

    output logic y   // Output
    );
//
endmodule
```

3. **Design the Carry-Out Logic 1-7:** Use the previously designed `nandN_delay` modules to implement the carry-out logic (i) for bits 1 to 7. See slide 9 in Lecture 4 to derive the equations for Carry-Out Logic 4, 5, 6, and 7

```
module carry_logic_i (
    input logic p0, pi,
    input logic g0, gi,
    input logic carry_in_{i-1},
    output logic carry_out_i);
//
endmodule
```

4. **Design and connect the 8-bit CLA:** Connect all modules (Propagate logic, Generate logic, Sum logic, and Carry out logic) to form the complete 8-bit signed two's complement Carry Look Ahead Adder. Do not forget to add sign extension logic to avoid overflow and overflow. See the 4-bit CLA model presented in the class.

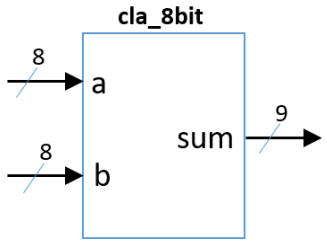


Figure 3- 8-bit CLA block

```
module cla_8bit (
    input  logic [7:0] a,
    input  logic [7:0] b,
    output logic [8:0] sum
);
//
endmodule
```

5. **Simulate and verify your 8-bit CLA:** Simulate your 8-bit CLA design to ensure correct functionality and verify the results. Make sure to include delay analysis in your simulation results to understand the timing characteristics of your 8-bit CLA design.

Bonus Point

1. **Modify the 8-bit CLA to 8-bit CLA with OVF (Overflow) and UVF (Underflow) detection (0.5 point):**

   Modify your designed 8-bit CLA to include overflow (OVF) and underflow (UVF) detection for signed two's complement arithmetic.

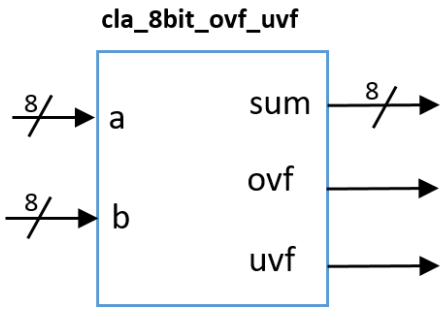


Figure 4- 8-bit CLA with OVF and UVF detection block

```
module cla_8bit_ovf_uvf (
    input  logic [7:0] a,
    input  logic [7:0] b,
    output logic [7:0] sum,
    output logic ovf, uvf
);
//
endmodule
```

2. **Add saturation mechanism (0.5 Point):**

   Modify your 8-bit CLA with OVF and UVF detection to include a saturation mechanism limiting the result to the maximum or minimum value in case of overflow or underflow.

Hint: You can use the Multiplexer structure, as was done in the carry select adder lecture.

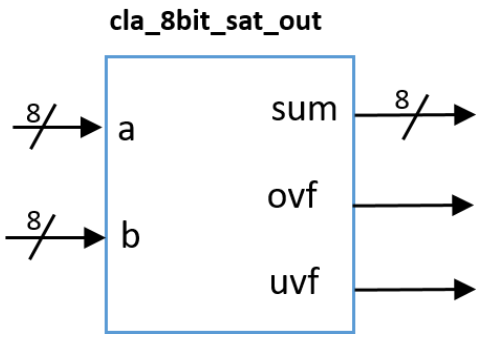


Figure 5- 8-bit CLA with saturated output

```
module cla_8bit_sat_out (
    input  logic [7:0] a,
    input  logic [7:0] b,
    output logic [7:0] sum,
    output logic ovf, uvf
);
//
endmodule
```

Guidelines

- Only structural style with port mapping is accepted (except nand2 gate).
- Keep in mind that you should use a *generate* structure when possible.
- Use the testbench we provided for you to test your 8-bit CLA module.